PROGRAMMING PARADIGMS

"I'm afraid you've had a paradigm shift."

Jason Atkin and Graham Hutton
University of Nottingham

Today

- Reflection on FP + Haskell;
- Reflection on OO + Java;
- Exam information.

REFLECTION ON FP + HASKELL

What Have You Learned?

- How to write simple Haskell programs;
- A new way of thinking;
- The power of types;
- The power of abstraction.

Key Concepts (1)

- Saying what to compute, rather than how:

```
sum . map (^2) . filter even
```

- Separating pure and impure code:

```
Int → Bool
```

vs

```
Int → IO Bool
```

Key Concepts (2)

- Functions as first-class citizens:

```
map :: (a → b) → [a] → [b]
```

- Equational reasoning about programs:

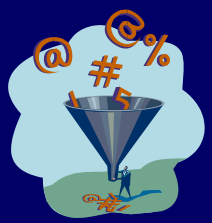
```
map f . map g  =  map (f . g)
```

Main Drawbacks

- Difficult to reason about efficiency;

- Limited tool support for developers;

- Requires ability to think abstractly.

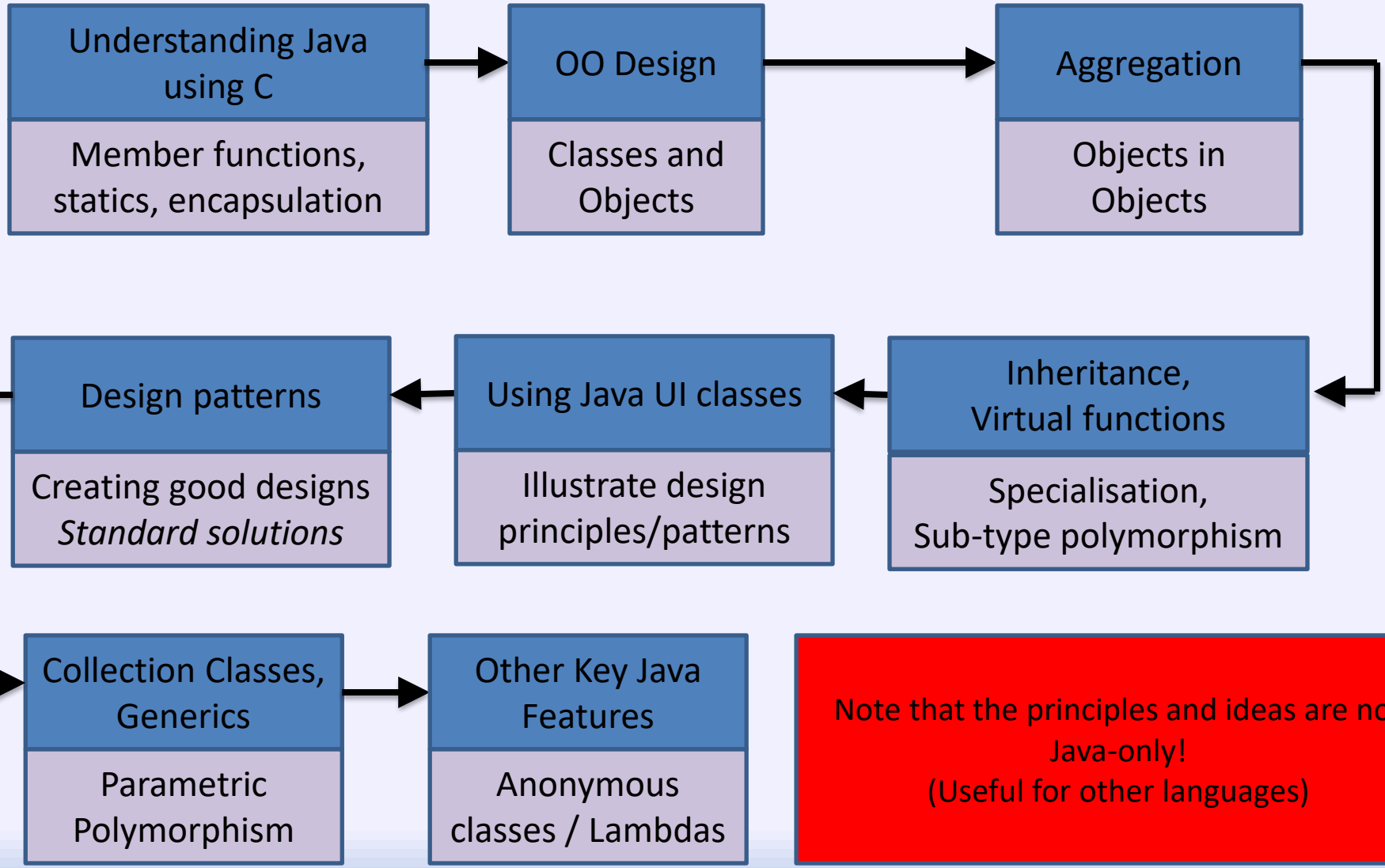


Taking Things Further

- Advanced Functional Programming;
- Other languages;
- Industry jobs;
- PhD positions.

REFLECTION ON OO + JAVA

Initial Outline



Key OO features that you should know

- Objects (and classes)
  - Objects are the things we create using new
  - Classes specify what is in an object and what we can do to them
- Encapsulation (methods and data together)
  - You put the methods which act on the data in the class
- Abstraction (and interfaces)
  - We only need to worry about what methods do, not how they are implemented
- Data hiding (restrict access to data)
  - Ideally we make data private and the 'interface' methods public
- Composition (stronger than aggregation)
  - Objects can contain other objects – and use them to do the work
- Inheritance (specialisation, reuse, 'is-a')
  - We used this a lot – reusing things like JButton and JLabel behaviour
- (Sub-type) Polymorphism (dynamic dispatch, late binding)
  - Behaviour only known at runtime – we used this a lot too
  - It's also the reason that Lambdas can 'wrap up' a function
- We also used generics (parametric polymorphism)
  - Which in Java are implemented using sub-type polymorphism

Java benefits for OO development

- Java code is compiled to class files
  - Java files can be compiled separately - .class files for each .java file
  - Class files are run within a java virtual machine
  - Java programs can be compiled on one machine and run on any others (copy the .class files and run them with 'java' program)
  - Write-once, run-anywhere (in theory)
- Java significantly simplifies (or removes) many things compared with OO languages like C++
  - Especially memory management
  - Next year's C++ course shows a more complex OO language
- Garbage collection is a great thing
  - Objects get destroyed when the last reference to them disappears (or when garbage collection is run after that)
  - Pointer errors and memory management are huge problems in C (using a pointer after the memory was freed, forgetting to free memory, freeing memory twice)
  - Note: C++ has 'smart pointers' to help with this

A few comparison comments…

- There is a lot more to Java than we have seen
  - Don't expect to be an expert already
  - We didn't cover the useful class libraries – we covered OO concepts
- We went through quite a lot, fairly quickly
  - But I limited it to only the key features and illustrative classes
  - But look how much you can already do (e.g. coursework)
- Graham kept to a small part of Haskell
- I must show you comparatively more Java
  - Consider what you achieved in Haskell vs Java coursework
  - Other modules need you to understand more Java as well as OO
- The benefits of OO are really only seen in larger programs
  - Especially in terms of reuse – of standard classes, e.g. GUI
  - We didn't have time for many standard classes though
- Java is **powerful** and **flexible** – get **reliable productivity**
  - There's a lot you can do, surprisingly easily, using existing classes

Some key differences: OO vs FP

- Everything is an object in the OO paradigm
  - Even when we emulate FP features we use objects (e.g. Lambdas)
  - In some OO languages (e.g. C++) objects can emulate pointers as objects as well (e.g. Smart Pointers)
- State is usually mutable (not 'final' / constant)
  - We often 'set' data in an object, changing the object's 'values'
    - Controlling what can set data is the key to OO decomposition
  - We *could* make things constant though (use 'final' in Java)
    - Or provide getters but not setters to make object immutable (e.g. String)
  - Data in F.P. is usually immutable
    - E.g. to reverse a list in FP you make a new list, vs Java modifying the list
- Many languages prefer iteration as being more efficient
  - FP tends to use recursion, and has optimisations for it
  - We could use recursion in Java (and other languages)
- Everything from Haskell could be done in Java
  - But it would be a **lot of work** to do it **manually** – not worth it
  - E.g. Haskell compiler handles things like lazy evaluation

My overview comparison

- Functional programming deliberately limits you – removing things that would 'break' features
- Because of this, the runtime can do a lot for you
  - Manage the order of evaluation for you
    - So, you are able to think in a **declarative** way
  - Execute a function once for each parameter value set
  - Lazy evaluation is a pain to do manually in Java
  - Handles deep recursion – avoiding stack overflow
- Object oriented programming adds facilities to:
  - Control data problems – e.g. manage data changes, decompose larger problems into independent parts
  - Facilitate code reuse – e.g. inheritance, aggregation

Exam Information

- In person, ExamSys exam;
- Two hours duration;
- Questions on both paradigms;
- Exam guides available on moodle.